Java Level 11 — DSA & Problem Solving

Part 2 — Binary Search & Linked Lists

Status

Level 11: In Progress

Covered in this part:

- ✅ Binary Search
- ✅ Search Insert Position
- ✅ First / Last Occurrence
- ✅ Search in Rotated Sorted Array
- ✅ Binary Search on Answer
- ✅ Reverse Linked List
- ✅ Slow & Fast Pointers
- ✅ Middle of Linked List
- ✅ Linked List Cycle Detection
- ✅ Dummy Node Pattern
- ✅ Merge Two Sorted Lists
- 🟡 Remove Nth Node From End — Concept covered, implementation pending

---

1. Binary Search

Binary Search efficiently searches an **ordered search space** by eliminating approximately half of the remaining possibilities during every iteration.

Given:

```
nums = [1,3,5,7,9,11]

target = 7
```

Instead of scanning:

```
1 → 3 → 5 → 7
```

we repeatedly inspect the middle.

Complexity

```
Time  → O(log n)
Space → O(1)
```

---

2. Basic Binary Search

Standard template:

```java
public static int binarySearch(int[] nums, int target) {
    int left = 0;
    int right = nums.length - 1;

    while (left <= right) {
        int mid = left + (right - left) / 2;

        if (nums[mid] == target) {
            return mid;
        } else if (nums[mid] < target) {
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }

    return -1;
}
```

Mental Model

```
nums[mid] == target
→ found

nums[mid] < target
→ target must be further right
→ left = mid + 1

nums[mid] > target
→ target must be further left
→ right = mid - 1
```

3. Important Mid Calculation

Prefer:

```
int mid = left + (right - left) / 2;
```

Do not use:

```
int mid = (right - left) / 2;
```

because this calculates only the offset from `left`.

Example:

```
left  = 4
right = 8

(right - left) / 2
= 2 ❌

left + (right - left) / 2
= 6 ✅
```

It also avoids potential overflow compared with:

```
(left + right) / 2
```

4. Search Insert Position

Problem:

```
nums = [1,3,5,6]

target = 5 → 2
target = 2 → 1
```

```
target = 7 → 4
target = 0 → 0
```

If the target isn't found, return the position where it should be inserted.

```java
public static int searchInsert(int[] nums, int target) {
    int left = 0;
    int right = nums.length - 1;

    while (left <= right) {
        int mid = left + (right - left) / 2;

        if (nums[mid] == target) {
            return mid;
        } else if (nums[mid] < target) {
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }

    return left;
}
```

Important Observation

When unsuccessful Binary Search finishes:

```
left
```

represents the insertion position.

So:

```java
return left;
```

not:

```java
return left + 1;
```

---

5. Binary Search Boundaries

Suppose:

```
nums = [1,2,2,2,3,4]
target = 2
```

Normal Binary Search may return any matching `2`.

Sometimes we specifically need:

```
First occurrence → 1
Last occurrence  → 3
```

This introduces **Boundary Binary Search**.

---

6. First Occurrence

When the target is found:

```
Don't return immediately.

Save answer
+
continue searching LEFT
```

```java
public static int firstOccurrence(int[] nums, int target) {
    int left = 0;
    int right = nums.length - 1;

    int answer = -1;

    while (left <= right) {
        int mid = left + (right - left) / 2;

        if (nums[mid] == target) {
            answer = mid;

            // Search for an earlier occurrence
            right = mid - 1;
```

```
        } else if (nums[mid] < target) {
            left = mid + 1;

        } else {
            right = mid - 1;
        }
    }

    return answer;
}
```

---

7. Last Occurrence

Same pattern, except when the target is found:

```
Save answer
+
continue searching RIGHT
```

```
public static int lastOccurrence(int[] nums, int target) {
    int left = 0;
    int right = nums.length - 1;

    int answer = -1;

    while (left <= right) {
        int mid = left + (right - left) / 2;

        if (nums[mid] == target) {
            answer = mid;

            // Search for a later occurrence
            left = mid + 1;

        } else if (nums[mid] < target) {
            left = mid + 1;

        } else {
            right = mid - 1;
        }
    }
```

```
    return answer;
}
```

Boundary Mental Model

```
First occurrence:

FOUND
→ save
→ LEFT


Last occurrence:

FOUND
→ save
→ RIGHT
```

Complexity:

```
Time  → O(log n)
Space → O(1)
```

---

8. Search in Rotated Sorted Array

Original:

```
[0,1,2,4,5,6,7]
```

Rotated:

```
[4,5,6,7,0,1,2]
```

We still want:

```
O(log n)
```

Critical Observation

Although the entire range may not be sorted:

> At least one half around `mid` is sorted.

First determine which half is sorted.

Left half sorted

```
nums[left] <= nums[mid]
```

Then determine whether the target belongs inside that range.

Otherwise

The right half is sorted.

---

9. Rotated Array Implementation

```java
public static int searchRotated(int[] nums, int target) {
    int left = 0;
    int right = nums.length - 1;

    while (left <= right) {
        int mid = left + (right - left) / 2;

        if (nums[mid] == target) {
            return mid;
        }

        // LEFT half is sorted
        if (nums[left] <= nums[mid]) {

            if (nums[left] <= target
                    && target < nums[mid]) {

                right = mid - 1;

            } else {
                left = mid + 1;
            }
```

```
        } else {

            // RIGHT half is sorted
            if (nums[mid] < target
                    && target <= nums[right]) {

                left = mid + 1;

            } else {
                right = mid - 1;
            }
        }
    }

    return -1;
}
```

Mental Model

```
Calculate mid
      ↓
Target found?
      ↓ no
Which half is sorted?
      ↓
Is target inside that sorted half?
      ↓
YES → search there

NO → search the other half
```

Do not blindly check whether the target belongs in the left range.

First determine:

```
Which half is sorted?
```

---

10. Binary Search on Answer

Binary Search does not have to search an array.

Sometimes we search:

```
possible answers
```

Common interview wording:

```
Find MINIMUM value such that...
Find MAXIMUM value such that...
Minimum capacity...
Minimum speed...
Maximum feasible...
```

If feasibility changes monotonically across the answer range, Binary Search may apply.

---

11. Koko Eating Bananas

Given:

```
piles = [3,6,7,11]
h = 8
```

Find the minimum eating speed.

At speed:

```
4 bananas/hour
```

hours required:

```
3  → 1
6  → 2
7  → 2
11 → 3

Total = 8
```

So:

```
answer = 4
```

12. Binary Search Search-Space

We're not searching:

```
piles
```

We're searching:

```
possible eating speeds

1 ... maxPile
```

For every candidate:

```
midSpeed
```

ask:

```
Can Koko finish within h hours?
```

If:

```
totalTime <= h
```

the speed works.

But we're finding the **minimum** speed:

```
save answer
search LEFT
```

If:

```
totalTime > h
```

the speed is too slow:

```
search RIGHT
```

---

13. Ceiling Division

For:

```
pile = 7
speed = 4
```

normal integer division gives:

```
7 / 4 = 1
```

But Koko actually requires:

```
2 hours
```

Integer ceiling division:

```
(pile + speed - 1) / speed
```

Safer with potential overflow:

```
((long) pile + speed - 1) / speed
```

---

14. Koko Implementation

```
public static int minEatingSpeed(int[] piles, int h) {
    int maxPile = 0;

    for (int pile : piles) {
        maxPile = Math.max(maxPile, pile);
    }

    int left = 1;
```

```
    int right = maxPile;

    int result = maxPile;

    while (left <= right) {
        int midSpeed =
                left + (right - left) / 2;

        long totalTime = 0;

        for (int pile : piles) {
            totalTime +=
                    ((long) pile + midSpeed - 1)
                            / midSpeed;
        }

        if (totalTime <= h) {

            // Feasible
            result = midSpeed;

            // Try smaller answer
            right = midSpeed - 1;

        } else {

            // Too slow
            left = midSpeed + 1;
        }
    }

    return result;
}
```

Complexity:

```
n = number of piles
m = largest pile

Time  → O(n log m)
Space → O(1)
```

---

15. Binary Search Recognition Cheat Sheet

Think Binary Search when you see:

```
Sorted array
Search target
Insertion position
First occurrence
Last occurrence
Find boundary
Rotated sorted array
Minimum feasible value
Maximum feasible value
```

Core question:

> Can the remaining search space be divided approximately in half?

For Binary Search on Answer:

> Is there a monotonic feasibility condition?

Example:

```
Speed too slow ❌
Speed too slow ❌
Speed works    ✅
Speed works    ✅
Speed works    ✅

               ↑
        Find this boundary
```

---

16. Linked Lists

Unlike an array:

```
[10][20][30][40]
```

a linked list stores nodes connected using references:

```
10 → 20 → 30 → 40 → null
```

Node:

```java
class ListNode {
    int val;
    ListNode next;

    ListNode(int val) {
        this.val = val;
    }
}
```

Important interview patterns:

```
Traversal
Pointer manipulation
Reversal
Slow/Fast pointers
Dummy nodes
Merging
Cycle detection
```

---

17. Reverse Linked List

Input:

```
10 → 20 → 30 → null
```

Output:

```
30 → 20 → 10 → null
```

We need:

```
prev
curr
next
```

Mental Model

```
SAVE
REVERSE
SHIFT
```

For every node:

```
1. Save curr.next

2. Reverse curr.next

3. Shift prev

4. Shift curr
```

---

18. Reverse Linked List Implementation

```java
public static ListNode reverseList(ListNode head) {
    ListNode prev = null;
    ListNode curr = head;

    while (curr != null) {
        ListNode next = curr.next;

        curr.next = prev;

        prev = curr;
        curr = next;
    }

    return prev;
}
```

Complexity:

```
Time  → O(n)
Space → O(1)
```

Important Pitfall

Wrong:

```
while (head != null)
```

if `head` never changes.

Use:

```
while (curr != null)
```

because `curr` is the traversal pointer.

---

19. Why Save `next` First?

Suppose:

```
10 → 20 → 30
```

If we immediately do:

```
curr.next = prev;
```

without saving the original `curr.next`, we can lose access to:

```
20 → 30
```

Therefore:

```
ListNode next = curr.next;
```

must happen first.

20. Slow & Fast Pointer Pattern

Initialize:

```
ListNode slow = head;
ListNode fast = head;
```

Move:

```
slow = slow.next;
fast = fast.next.next;
```

Meaning:

```
slow → 1 step

fast → 2 steps
```

Useful for:

```
Middle of Linked List
Cycle Detection
Cycle Starting Point
Palindrome Linked List
```

---

21. Middle of Linked List

Given:

```
1 → 2 → 3 → 4 → 5
```

return:

```
3
```

For:

```
1 → 2 → 3 → 4 → 5 → 6
```

return second middle:

```
4
```

Implementation:

```java
public static ListNode middleNode(ListNode head) {
    ListNode slow = head;
    ListNode fast = head;

    while (fast != null
            && fast.next != null) {

        slow = slow.next;
        fast = fast.next.next;
    }

    return slow;
}
```

Mental model:

```
fast travels twice as fast

When fast reaches the end:

slow ≈ halfway
```

Complexity:

```
Time  → O(n)
Space → O(1)
```

---

22. Linked List Cycle Detection

A cycle:

```
1 → 2 → 3 → 4
        ↑     ↓
        ← ← ←
```

Use the same:

```
slow = 1 step
fast = 2 steps
```

If there is no cycle:

```
fast eventually reaches null
```

If there is a cycle:

```
fast eventually catches slow
```

---

23. Floyd's Cycle Detection

```java
public static boolean hasCycle(ListNode head) {
    ListNode slow = head;
    ListNode fast = head;

    while (fast != null
            && fast.next != null) {

        slow = slow.next;
        fast = fast.next.next;

        if (slow == fast) {
            return true;
        }
    }

    return false;
}
```

Complexity:

```
Time  → O(n)
Space → O(1)
```

Important Pitfall

Do not wait until after the loop to check:

```
return slow == fast;
```

For a cycle, the pointers need to be compared **inside** the traversal.

```
if (slow == fast) {
    return true;
}
```

---

24. Dummy Node Pattern

Dummy nodes simplify linked-list operations where the head may change.

```
ListNode dummy = new ListNode(0);
dummy.next = head;
```

Think:

```
dummy → actual list
```

Benefits:

```
Avoid special handling for first node
Simplify head deletion
Simplify list construction
Simplify merging
```

A useful rule:

> If the head might be replaced or deleted, consider a dummy node.

---

25. Merge Two Sorted Lists

Given:

```
list1:

1 → 2 → 4

list2:

1 → 3 → 4
```

Result:

```
1 → 1 → 2 → 3 → 4 → 4
```

Create:

```
ListNode dummy = new ListNode(0);
ListNode curr = dummy;
```

Important distinction:

```
dummy
→ permanent reference to beginning

curr
→ where we're currently building
```

---

26. Merge Implementation

```
public static ListNode mergeTwoLists(
        ListNode list1,
        ListNode list2
) {
    ListNode dummy = new ListNode(0);
    ListNode curr = dummy;
```

```
    while (list1 != null
            && list2 != null) {

        if (list1.val <= list2.val) {

            curr.next = list1;
            list1 = list1.next;

        } else {

            curr.next = list2;
            list2 = list2.next;
        }

        curr = curr.next;
    }

    curr.next =
            list1 != null
                    ? list1
                    : list2;

    return dummy.next;
}
```

Complexity:

```
n = list1 length
m = list2 length

Time  → O(n + m)
Space → O(1)
```

---

27. Important Merge Mistakes

Wrong loop condition

Avoid:

```
while (list1.next != null
        && list2.next != null)
```

This can:

```
skip final nodes
cause NullPointerException
```

Use:

```
while (list1 != null
        && list2 != null)
```

Move the consumed list

After:

```
curr.next = list1;
```

you must:

```
list1 = list1.next;
```

Similarly for `list2`.

Preserve dummy

Don't keep moving the only `dummy` reference.

Instead:

```
ListNode dummy = new ListNode(0);
ListNode curr = dummy;
```

Move:

```
curr
```

and finally:

```
return dummy.next;
```

---

28. Remove Nth Node From End

Status: 🟡 Concept Covered — Implementation Pending

Problem:

```
1 → 2 → 3 → 4 → 5

n = 2
```

Remove second node from the end:

```
1 → 2 → 3 → 5
```

This combines:

```
Dummy Node
+
Two Pointers
```

---

29. Remove Nth Node Mental Model

Create:

```
ListNode dummy = new ListNode(0);
dummy.next = head;

ListNode slow = dummy;
ListNode fast = dummy;
```

Then:

```
1. Move fast n + 1 positions
```

```
2. Move slow and fast together

3. When fast == null:

   slow is immediately BEFORE
   the node we need to remove

4. Delete:

   slow.next = slow.next.next
```

Why the gap?

```
fast stays n + 1 nodes ahead
```

Therefore when `fast` passes the end, `slow` is positioned before the target.

---

30. Why Dummy Helps Here

Consider:

```
1 → 2

n = 2
```

We're deleting the head.

Result:

```
2
```

Dummy gives:

```
dummy → 1 → 2
```

Now deleting `1` is no different from deleting any other node:

```
slow.next = slow.next.next;
```

Finally:

```
return dummy.next;
```

Implementation will be completed during the next practice/revision.

---

31. Linked List Recognition Cheat Sheet

Reverse

```
Reverse linked list
Reverse portion
Rewire pointers
```

Think:

```
prev
curr
next

SAVE
REVERSE
SHIFT
```

Slow + Fast

```
Middle
Cycle
Cycle start
Palindrome
```

Think:

```
slow → 1
fast → 2
```

Dummy Node

```
Delete node
Head may change
Merge lists
Build new list
Partition list
```

Think:

```
dummy = permanent fake beginning
curr  = working pointer
```

Two Pointers With Gap

```
Nth node from end
Kth node from end
```

Think:

```
Move fast ahead

Maintain fixed gap
```

---

32. Common Mistakes From Practice

Binary Search Midpoint

Wrong:

```
int mid = (right - left) / 2;
```

Correct:

```
int mid =
        left + (right - left) / 2;
```

Search Insert Position

Wrong:

```
return left + 1;
```

Correct:

```
return left;
```

When Binary Search finishes unsuccessfully, `left` represents the insertion boundary.

Rotated Binary Search

Don't immediately ask:

```
Is target in left range?
```

First ask:

```
Which half is sorted?
```

Then:

```
Is target inside that sorted half?
```

Reverse Linked List

Loop using the pointer actually being advanced:

```
while (curr != null)
```

not an unchanged `head`.

Slow / Fast Pointer

Don't introduce an unnecessary third traversal pointer.

Use:

```
slow = slow.next;
fast = fast.next.next;
```

The pointers themselves maintain their positions.

Cycle Detection

Compare:

```
slow == fast
```

inside the loop after advancing them.

Merge Lists

Remember all three movements:

```
Attach selected node

Move selected input list

Move curr
```

---

33. Complexity Cheat Sheet

Problem / Pattern	Time	Space
Basic Binary Search	`O(log n)`	`O(1)`
Search Insert	`O(log n)`	`O(1)`
First / Last Occurrence	`O(log n)`	`O(1)`
Rotated Array Search	`O(log n)`	`O(1)`
Koko / Binary Search Answer	`O(n log m)`	`O(1)`
Reverse Linked List	`O(n)`	`O(1)`
Middle Node	`O(n)`	`O(1)`
Cycle Detection	`O(n)`	`O(1)`
Merge Sorted Lists	`O(n + m)`	`O(1)`
Remove Nth From End	`O(n)`	`O(1)`

---

34. Revision Checklist

Binary Search

- [x] Basic Binary Search
- [x] Search Insert Position
- [x] First Occurrence
- [x] Last Occurrence
- [x] Search Rotated Sorted Array
- [x] Binary Search on Answer
- [x] Koko Eating Bananas

Linked List

- [x] Reverse Linked List
- [x] Slow / Fast Pointer
- [x] Middle of Linked List
- [x] Cycle Detection
- [x] Dummy Node Pattern
- [x] Merge Two Sorted Lists
- [ ] Remove Nth Node From End — implementation pending

---

35. Level 11 Progress

PART 1

✅ Big-O

✅ Arrays & Hashing

✅ Two Pointers

✅ Sliding Window

✅ Prefix Sum

✅ Stack

✅ Monotonic Stack

✅ Queue / Deque Basics

🟡 Monotonic Deque — revisit

PART 2

✅ Binary Search

✅ Binary Search Boundaries

✅ Rotated Binary Search

✅ Binary Search on Answer

✅ Linked List Fundamentals

✅ Reverse Linked List

✅ Slow / Fast Pointers

✅ Cycle Detection

✅ Dummy Node

✅ Merge Sorted Lists

🟡 Remove Nth From End — implementation pending

```
NEXT

⬜ Trees / BST

⬜ Heap / Priority Queue

⬜ Graphs

⬜ BFS / DFS

⬜ Topological Sort

⬜ Union Find

⬜ Recursion

⬜ Backtracking

⬜ Greedy

⬜ Dynamic Programming

⬜ Intervals
```

Final Mental Model

Binary Search

```
Ordered search space?
        ↓
Can I discard half?
        ↓
Binary Search
```

For boundaries:

```
FOUND
but need earlier?
→ search LEFT
```

```
FOUND
but need later?
→ search RIGHT
```

For answer-space problems:

```
Possible answer
      ↓
Feasible?
   /
 YES       NO
 ↓          ↓
try         move toward
better      feasibility
answer
```

Linked Lists

Don't think of Linked List problems primarily as values.

Think:

```
REFERENCES
+
POINTER MOVEMENT
```

Most problems reduce to a few patterns:

```
Reverse
→ prev / curr / next

Middle / Cycle
→ slow / fast

Head may change
→ dummy node

Nth from end
→ fixed pointer gap
```

```
Merge
→ dummy + curr + input pointers
```

The goal is not to memorize individual solutions.

The goal is to recognize:

```
"What relationship between pointers
would make this problem easy?"
```

That is the core Linked List interview skill.